CS 300 ABCU Project One
Joshua Sevy
April 8, 2026

This project compares three different data structures that ABC University (ABCU) could use to store and manage course data: a vector, a hash table, and a binary search tree. Each design reads course data from a file, validates the data, creates course objects, and supports the two main advising requirements: printing all courses in alphanumeric order and printing the title and prerequisites for an individual course. After outlining the pseudocode for each approach, this project evaluates the runtime and memory tradeoffs of each structure and recommends the one that best fits the project requirements.

Section 1: Helper Functions Pseudocode

Validate Course Line Function (shared):

```
FUNCTION isValidCourseLine(tokens)
      // Every course must have at least a course number and title.
      IF number of tokens < 2
            RETURN FALSE
      END IF
      // Required fields should not be empty
      IF tokens[0] is EMPTY OR tokens[1] is EMPTY
            RETURN FALSE
      END IF
      RETURN TRUE
END FUNCTION
```

Prerequisite Exists Function (shared):

```
FUNCTION prerequisiteExists(prereq, validCourseNumbers)
      FOR EACH courseNumber in validCourseNumbers
            IF courseNumber is equal to prereq
                  RETURN TRUE
            END IF
      END FOR EACH
      RETURN FALSE
END FUNCTION
```

Hash Function (hash table Only):

```
FUNCTION hash(courseNumber)
      // Convert the course number into an integer value
      // and map it to a bucket in the table
      RETURN integer value MODULO tableSize
END FUNCTION
```

Section 2: Menu Pseudocode

```
FUNCTION main()
      SET choice = 0
      WHILE choice is not equal to 9
            PRINT "Menu:"
            PRINT "1. Load Data Structure"
            PRINT "2. Print Course List"
            PRINT "3. Print Course"
            PRINT "9. Exit"
            GET choice
            // Use a switch statement to handle menu options cleanly
            SWITCH choice
                  CASE 1
                        GET fileName
                        CALL loadCoursesFromFile(fileName, dataStructure)
                        BREAK
                  CASE 2
                        CALL printCourseList(dataStructure)
                        BREAK
                  CASE 3
                        GET courseNumber
                        CALL printCourseInformation(
                                dataStructure,
                                courseNumber
                              )
                        BREAK
                  CASE 9
                        PRINT "Goodbye."
                        BREAK
                  DEFAULT
                        PRINT "Invalid choice."
                        BREAK
            END SWITCH
      END WHILE
END FUNCTION
```

Section 3: Vector Pseudocode

Load Courses from File to Vector:

```
FUNCTION loadCoursesFromFile(fileName, courses)
      OPEN fileName
      IF file cannot be opened
            PRINT “Error: unable to open file.”
            RETURN
      END IF
      CREATE empty VECTOR<STRING> rawLines
      CREATE empty VECTOR<STRING> validCourseNumbers

      // First pass:
      // Use a two-pass approach so prerequisites can be validated
      // after all course numbers are known
      // Read each line, validate the minimum format,
      // and collect all course numbers for later prerequisite checks
      WHILE file has another line
            READ line
            IF line is EMPTY
                  PRINT “Skipping empty line.”
                  CONTINUE
            END IF
            SPLIT line by comma into tokens
            IF isValidCourseLine(tokens) is FALSE
                  PRINT “Invalid file format: “ + line
                  CONTINUE
            END IF
            ADD line to rawLines
            ADD tokens[0] to validCourseNumbers
      END WHILE
      CLOSE file

      // Second pass:
      // Build course objects and verify that each prerequisite
      // exists somewhere in the file.
      FOR EACH line in rawLines
            SPLIT line by comma into tokens
```

```
            CREATE Course newCourse
            SET newCourse.courseNumber = tokens[0]
            SET newCourse.courseTitle = tokens[1]

            CREATE empty VECTOR<STRING> prereqList
            SET isValid = TRUE

            FOR i = 2 to number of tokens – 1
                  SET prereq = tokens[i]

                  IF prerequisiteExists(
                        prereq,
                        validCourseNumbers
                     ) is FALSE
                        PRINT "Invalid prerequisite "
                              + prereq + " for course "
                              + newCourse.courseNumber
                        SET isValid = FALSE
                        BREAK
                  END IF
                  ADD prereq to prereqList
            END FOR

            // Only store valid courses so bad data does not affect the
            // final list
            IF isValid is TRUE
                  SET newCourse.prerequisites = prereqList
                  ADD newCourse to courses
            ELSE
                  // Invalid courses are skipped and not stored
                  CONTINUE
            END IF
      END FOR EACH
END FUNCTION
```

Print All Courses in the Vector:

```
FUNCTION printCourseList(courses)
      // Vector requires sorting before printing
      SORT courses by courseNumber
      FOR EACH course in courses
            PRINT course.courseNumber + ", " + course.courseTitle
      END FOR EACH
END FUNCTION
```

Search the Vector for a Single Course and Print the Course Information:

```
FUNCTION searchCourse(courses, courseNumber)
      FOR EACH course in courses
            IF course.courseNumber equals courseNumber
                  PRINT course.courseNumber + “, “ + course.courseTitle
                  IF course.prerequisites is EMPTY
                        PRINT “Prerequisites: None”
                  ELSE
                        PRINT “Prerequisites: “
                        FOR EACH prereq in course.prerequisites
                              PRINT prereq
                        END FOR EACH
                  END IF
                  RETURN
            END IF
      END FOR EACH
      PRINT “Course not found.”
END FUNCTION
```

Section 4: Hash Table Pseudocode

Insert Course into Hash Table:

```
FUNCTION insertCourse(hashTable, course)
      SET key to hash(course.courseNumber)
      IF hashTable.at(key) is EMPTY
            CREATE newNode with course
            SET hashTable.at(key) to newNode
      ELSE
            SET current to hashTable.at(key)
            WHILE current.next is NOT null
                  SET current to current.next
            END WHILE
            CREATE newNode with course
            SET current.next to newNode
      END IF
END FUNCTION
```

Load Courses from File to Hash Table:

```
FUNCTION loadCoursesFromFile(fileName, hashTable)
      OPEN fileName
      IF file cannot be opened
            PRINT “Error: unable to open file.”
            RETURN
      END IF
      CREATE EMPTY VECTOR<STRING> rawLines
      CREATE EMPTY VECTOR<STRING> validCourseNumbers
      // First pass:
      // Use a two-pass approach so prerequisites can be validated
      // after all course numbers are known
      // Read each line, validate the minimum format,
      // and collect all course numbers for a later prerequisite check
      WHILE file has another line
            READ line
            IF line is EMPTY
                  PRINT “Skipping empty line.”
                  CONTINUE
            END IF
            SPLIT line by comma into tokens
            IF isValidCourseLine(tokens) is FALSE
                  PRINT “Invalid file format: “ + line
                  CONTINUE
            END IF
            ADD line to rawLines
            ADD tokens[0] to validCourseNumbers
      END WHILE
      CLOSE file
      // Second pass:
      // Build course objects and verify that each prerequisite
      // exists somewhere in the file.
      FOR EACH line in rawLines
            SPLIT line by comma into tokens
            CREATE Course newCourse
            SET newCourse.courseNumber = tokens[0]
            SET newCourse.courseTitle = tokens[1]
            CREATE EMPTY VECTOR<STRING> prereqList
            SET isValid = TRUE
            FOR i = 2 to number of tokens – 1
                  SET prereq = tokens[i]
                  IF prerequisiteExists(
                        prereq,
                        validCourseNumbers
```

```
                    ) is FALSE
                       PRINT "Invalid prerequisite "
                             + prereq
                             + " for course "
                             + newCourse.courseNumber
                       SET isValid = FALSE
                       BREAK
                 END IF
                 ADD prereq to prereqList
            END FOR
            // Only store valid courses in the hash table
            IF isValid is equal to TRUE
                 SET newCourse.prerequisites = prereqList
                 CALL insertCourse(hashTable, newCourse)
            ELSE
                 // Invalid courses are skipped and not stored
                 CONTINUE
            END IF
      END FOR EACH
END FUNCTION
```

Search the Hash Table:

```
FUNCTION searchCourse(hashTable, courseNumber)
      SET key to hash(courseNumber)
      SET current to hashTable.at(key)
      WHILE current is NOT null
            IF current.course.courseNumber is equal to courseNumber
                 RETURN current.course
            END IF
            SET current to current.next
      END WHILE
      RETURN EMPTY
END FUNCTION
```

Print Course Information and Prerequisites:

```
FUNCTION printCourseInformation(hashTable, courseNumber)
      SET course to searchCourse(hashTable, courseNumber)
      IF course.courseNumber is EMPTY
            PRINT "Course not found."
            RETURN
      END IF
```

```
    PRINT course.courseNumber + “, “ + course.courseTitle
    IF course.prerequisites is EMPTY
        PRINT “Prerequisites: None”
    ELSE
        PRINT “Prerequisites: “
        FOR EACH prereq in course.prerequisites
            PRINT prereq
        END FOR EACH
    END IF
END FUNCTION
```

Print All Courses in Alphanumeric Order:

```
FUNCTION printCourseList(hashTable)
    // Hash table does not maintain order, so values must be
    // collected and sorted
    CREATE EMPTY VECTOR<Course> sortedCourses
    // Walk each bucket in the hash table
    FOR EACH bucket in hashTable
        SET current to bucket
        WHILE current is NOT null
            ADD current.course to sortedCourses
            SET current to current.next
        END WHILE
    END FOR
    // Sort collected courses by courseNumber
    SORT sortedCourses by courseNumber
    // Print the sorted result
    FOR EACH course in sortedCourses
        PRINT course.courseNumber + “, “ + course.courseTitle
    END FOR
END FUNCTION
```

Section 5: Binary Search Tree Pseudocode

Insert Course into Binary Search Tree:

```
FUNCTION insertCourse(root, course)
    IF root is null
        CREATE newNode with course
        SET root to newNode
        RETURN root
    END IF
```

```
    IF course.courseNumber < root.course.courseNumber
        root.left = insertCourse(root.left, course)
    ELSE
        root.right = insertCourse(root.right, course)
    END IF
    RETURN root
END FUNCTION
```

Load Courses from File to Binary Search Tree:

```
FUNCTION loadCoursesFromFile(fileName, root)
    OPEN fileName
    IF file cannot be opened
        PRINT "Error: unable to open file."
        RETURN root
    END IF
    CREATE EMPTY VECTOR<STRING> rawLines
    CREATE EMPTY VECTOR<STRING> validCourseNumbers
    // First pass:
    // Use a two-pass approach so prerequisites can be validated
    // after all course numbers are known
    // Read each line, validate the minimum format,
    // and collect all course numbers for a later prerequisite check
    WHILE file has another line
        READ line
        IF line is EMPTY
            PRINT "Skipping empty line."
            CONTINUE
        END IF
        SPLIT line by comma into tokens
        IF isValidCourseLine(tokens) is FALSE
            PRINT "Invalid file format: " + line
            CONTINUE
        END IF
        ADD line to rawLines
        ADD tokens[0] to validCourseNumbers
    END WHILE
    CLOSE file
    // Second pass:
    // Build course objects and verify that each prerequisite
    // exists somewhere in the file.
    FOR EACH line in rawLines
        SPLIT line by comma into tokens
        CREATE Course newCourse
```

```
            SET newCourse.courseNumber = tokens[0]
            SET newCourse.courseTitle = tokens[1]
            CREATE EMPTY VECTOR<STRING> prereqList
            SET isValid = TRUE
            FOR i = 2 to number of tokens – 1
                  SET prereq = tokens[i]
                  IF prerequisiteExists(
                              prereq,
                              validCourseNumbers
                     ) is FALSE
                        PRINT (
                              "Invalid prerequisite "
                              + prereq
                              + " for course "
                              + newCourse.courseNumber
                              )
                        SET isValid = FALSE
                        BREAK
                  END IF
                  ADD prereq to prereqList
            END FOR
            // Only store valid courses, so bad data does not impact
            // the binary search tree
            IF isValid is equal to TRUE
                  SET newCourse.prerequisites = prereqList
                  SET root = insertCourse(root, newCourse)
            ELSE
                  // Invalid courses are skipped and not stored
                  CONTINUE
            END IF
      END FOR EACH
      RETURN root
END FUNCTION
```

Search the Binary Search Tree:

```
FUNCTION searchCourse(root, courseNumber)
      SET current to root
      WHILE current is NOT null
            IF current.course.courseNumber is equal to courseNumber
                  RETURN current.course
            END IF
            IF courseNumber < current.course.courseNumber
                  SET current to current.left
```

```
            ELSE
                  SET current to current.right
            END IF
      END WHILE
      RETURN EMPTY
END FUNCTION
```

Print Course Information and Prerequisites:

```
FUNCTION printCourseInformation(root, courseNumber)
      SET course to searchCourse(root, courseNumber)
      IF course.courseNumber is EMPTY
            PRINT "Course not found."
            RETURN
      END IF
      PRINT course.courseNumber + ", " + course.courseTitle
      IF course.prerequisites is EMPTY
            PRINT "Prerequisites: None"
      ELSE
            PRINT "Prerequisites: "
            FOR EACH prereq in course.prerequisites
                  PRINT prereq
            END FOR EACH
      END IF
END FUNCTION
```

Print All Courses in Order:

```
FUNCTION printCourseList(root)
      // BST naturally prints in order using in-order traversal
      IF root is NOT null
            CALL printCourseList(root.left)
            PRINT root.course.courseNumber
                  + ", "
                  + root.course.courseTitle
            CALL printCourseList(root.right)
      END IF
END FUNCTION
```

Section 6: Runtime and Memory Analysis

Data Structure	File Read + Validation	Course Object Creation	Worst-case Runtime	Memory Notes
Vector	Two passes through file plus prerequisite checks	Append to vector	$O(n^2)$	Stores all course objects plus prerequisite lists. Low overhead beyond the vector itself.
Hash Table	Two passes through file plus prerequisite checks	Average insertion $O(1)$, worst-case chaining $O(n)$	$O(n^2)$	Stores course object plus hash buckets and linked nodes for collisions. More overhead than vector.
Binary Search Tree	Two passes through file plus prerequisite checks	Average insertion $O(log\ n)$, worst-case unbalanced $O(n)$	$O(n^2)$	Stores course objects plus tree nodes and child pointers. More overhead than vector but supports ordered traversal naturally.

For all three approaches, the slowest part is the two-pass file validation process. In the first pass, the program reads each line and collects all course numbers, which takes $O(n)$ time. In the second pass, it reads the stored lines again, creates course objects, and checks each prerequisite against the list of course numbers. Since checking prerequisites means scanning the list, this step can take $O(n)$ for each course in the worst case. So, the total loading process can reach $O(n^2)$ for the vector, hash table, and tree versions.

The differences between these data structures stand out more after loading the data. Adding a course to a vector is easy, but finding a specific course takes linear time unless the data is sorted and searched efficiently. Hash tables let you look up courses quickly on average, but they do not keep courses in order, so printing a sorted list takes extra steps. Binary search trees make searching more efficient than vectors and allow in-order traversal, which makes it easier to

print courses in alphanumeric order. Because all three approaches share the same validation process, their loading performance is similar, but their efficiency differs when performing sorting and searching operations after the data has been loaded.

Section 7: Advantages and Disadvantages

Vector

A vector is the easiest structure to build and understand. It works well for loading course objects and is easy to sort when printing all courses in order. The design is simple, which also makes debugging easier. However, searching for a specific course means scanning through the whole vector unless another search method is added. This becomes less efficient as the number of courses increases.

Hash Table

A hash table is strong when the main goal is fast lookup of an individual course. Once the data is loaded, a course can usually be found quickly by hashing the course number and checking the correct bucket. The main disadvantage is that hash tables do not keep data in sorted order. Because of that, printing all courses alphabetically requires collecting the values, sorting them separately, and then printing them. It also has more memory overhead due to buckets and chained nodes.

Binary Search Tree

A binary search tree provides a good balance between search efficiency and ordered output. Searching for a course is more efficient than scanning a vector and printing all courses in order can be done naturally with an in-order traversal. This fits the advisor requirements well.

The disadvantage is that insertion and search can degrade in the worst case if the tree becomes unbalanced, and the structure is more complex to implement than a vector.

Section 8: Recommendation

Given the project requirements, I would recommend implementing a binary search tree for the final project. The three possible implementations discussed above can store the set of courses. However, the binary search tree meets the advisor's needs better than the other two due to its ability to perform in-order traversal, which will help in obtaining an alphabetically sorted list without additional actions. Additionally, searching for a particular course takes less time in a binary search tree compared to a vector.

A hash table is convenient for searching data, but it does not preserve the sequence. Therefore, certain actions should be taken before obtaining the complete alphabetical list of courses. A vector is the easiest way to work with the courses; however, it is less efficient for searching courses. All three methods require the same amount of time for loading the data $O(n^2)$, as the check for prerequisites will occur anyway. Given that the project requires printing courses in alphabetical order and allowing searches for a particular one, it would be optimal to use a binary search tree.